STATEMENT OF WORK · BUILD PLAN

LLM Feature Evaluation & Quality Gate

A representative engagement: take one AI feature you're not yet confident shipping, build the evaluation and CI gate that keeps it honest, and hand you a repeatable quality system your team can run without me. This document shows exactly how I scope, build, and prove the work.

> **Sample document.** This is an illustrative build plan, not a binding quote. Real scope, timeline, and price are confirmed after a short call and written into a project-specific SOW. Figures below are indicative ranges, not fixed prices.

1 Parties & engagement

Provider	Sage Ideas LLC — Jason Teixeira (hello@sageideas.dev)
Client	[Client legal name]
Engagement type	Fixed-scope project · one AI feature · your repository
Estimated duration	~4 weeks from kickoff (see §5)
Working model	Remote · async-first with a weekly checkpoint call · your stack, your git

2 Objective & success criteria

The problem this solves. Most teams ship an AI feature and then hope it behaves. "It seems to work" is not a release criterion — and it fails quietly, in front of customers, the first time a prompt, a model version, or a retrieval index drifts.

What "done" means here. The engagement is successful when every one of these is true and demonstrable:

- A **golden evaluation set** exists for the feature, with pass/fail scoring anyone on your team can read.
- A **CI gate** blocks a merge when quality, safety, or cost regresses past a threshold you set.
- The feature ships with a **safety battery** covering hallucination, prompt injection, PII leakage, and toxicity.
- Your team can **run and extend the whole system** from a documented runbook — no dependency on me.

3 Scope of work

Delivered in four phases, each ending in something you can see and verify — not a status update.

01 Discovery & risk map

~WEEK 1

Review the feature, its prompts, data flow, and failure modes. Agree the quality dimensions that matter (accuracy, grounding, safety, latency, cost) and the thresholds that define "shippable." Produce a one-page risk model so coverage tracks real risk, not guesswork.

DELIVERABLE

Risk model + evaluation plan (agreed thresholds)

02 Build the evaluation harness

~WEEKS 1–2

Author the golden set from real and adversarial inputs. Implement scoring (deterministic checks + LLM-as-judge where judgment is needed), the safety battery, and per-run quality/latency/cost measurement. Every check is version-controlled in your repo.

DELIVERABLE

Golden set + scoring harness + safety battery, in your repo

03 Wire the CI gate

~WEEK 3

Connect the harness to your CI so a pull request that regresses quality, safety, or cost is blocked with a readable report. Add a ratchet so the bar only moves up. Prove it works by making it fail on purpose, then pass.

DELIVERABLE

Passing CI gate + a captured red→green run proving it fires

04 Handoff & enablement

~WEEK 4

Write the runbook: how to add a case, read a failure, tune a threshold, and interpret the gate. Walk your team through it live. The goal is that the system outlives the engagement and needs no consultant to operate.

DELIVERABLE

Runbook + recorded walkthrough + handoff session

4 Deliverables summary

DELIVERABLE	FORM	ACCEPTANCE
Risk model & evaluation plan	Document, agreed thresholds	Signed off in the week-1 checkpoint
Golden evaluation set	Versioned data in your repo	Covers the agreed dimensions
Scoring harness + safety battery	Code in your repo	Runs locally & in CI, green
CI quality gate	CI workflow	Blocks a seeded regression; passes clean
Runbook + walkthrough	Docs + recording	Your team runs it unaided

5 Timeline

WEEK	FOCUS	CHECKPOINT
1	Discovery, risk map, evaluation plan; start the golden set	Plan sign-off
2	Harness, scoring, safety battery	Harness demo
3	CI gate + ratchet; prove it fires	Red→green captured
4	Runbook, enablement, handoff	Final acceptance

Timeline assumes timely access (repo, CI, a representative environment) and one client point of contact for decisions. Slippage in access shifts the schedule, not the scope.

6 Out of scope

To keep the engagement fixed and honest, the following are explicitly excluded unless added in writing: building the underlying feature itself; model training or fine-tuning; production infrastructure changes; ongoing on-call or monitoring (available separately as a retainer); and evaluation of features beyond the one named in §1.

7 Assumptions & client responsibilities

- Timely access to the repository, CI system, and a representative test environment.
- One empowered point of contact for scope and threshold decisions.
- Existing feature is functional enough to evaluate (this engagement measures and gates it, it does not build it).
- Any third-party API costs incurred during evaluation are the client's (typically minimal).

8 Commercials

This engagement is quoted as a fixed price after a scoping call — you approve the number before any work starts, in writing. Indicative range for a single-feature evaluation & gate:

COMPONENT	MODEL	INDICATIVE
Eval Audit (entry point)	Fixed · credits into the build	`from $2,500`
Full evaluation & CI gate build	Fixed-scope project	`$8k–$18k`
Ongoing quality retainer	Monthly · optional	`$2.5k–$4k / mo`

Indicative ranges for illustration only; not an offer. Deposit of 30% books the engagement; balance on final acceptance. Exact figures are set in the project SOW after a call.

9 Next step

One 30-minute call turns this sample into a real, fixed quote for your feature — with a plan you keep either way.

`agency.sageideas.dev/book.html`

Jason Teixeira
Sage Ideas LLC · Provider

[Client signatory]
[Client legal name] · Client

Signature & date

Signature & date

This sample is provided for illustration and does not itself create a binding agreement. A project-specific Statement of Work, executed under a Master Services Agreement, governs any engagement.